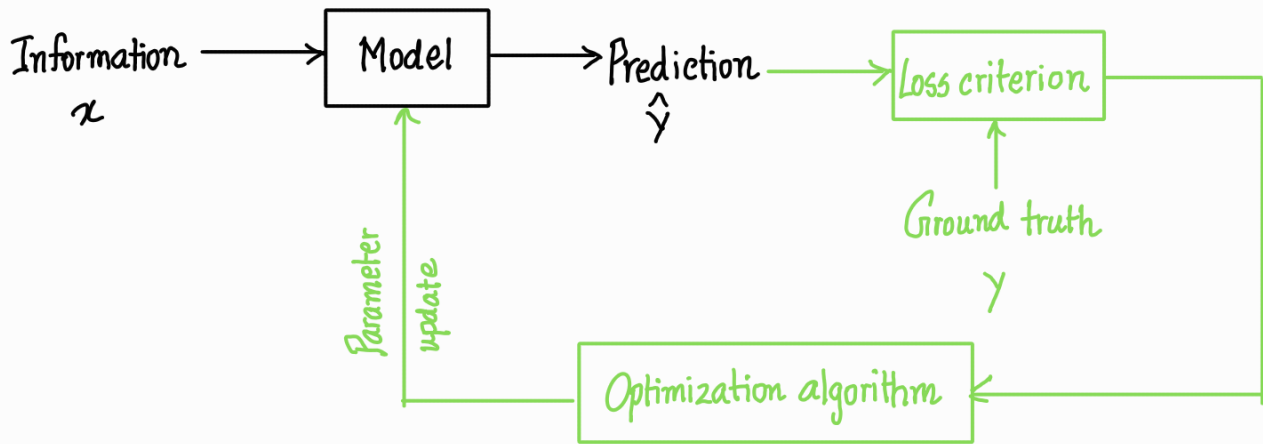


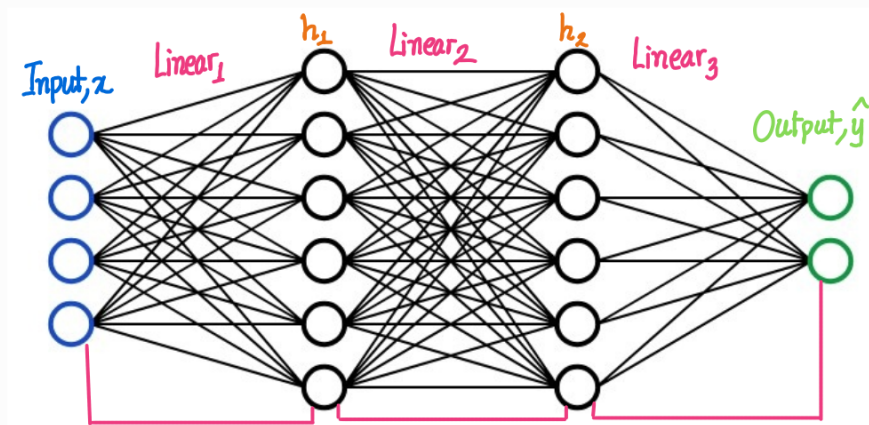
You now know about simple learning systems. The general pattern of a learning system —



Green paths only exist during training.

Now, the most common learning system is a neural network composed of linear layers.

You have already learnt a bit about linear layers in the lab.



Linear layers are also called fully connected (FC) layers

A linear layer is just a matrix formed with hyperparameters — n (in_features) and n (out_features). We will assume a bias is always present.

In the given picture, $\text{Linear}_1 = \text{Linear}(4, 6)$,

$\text{Linear}_2 = \text{Linear}(6, 6)$,

$\text{Linear}_3 = \text{Linear}(6, 2)$

then $h_1 = \text{Linear}_1(x) = x \overset{\text{matmul}}{\omega_1^T} + b_1 \leftarrow \text{bias [out-features] [6]}$

$\uparrow$
weight [out_features, in_features] [6, 4]

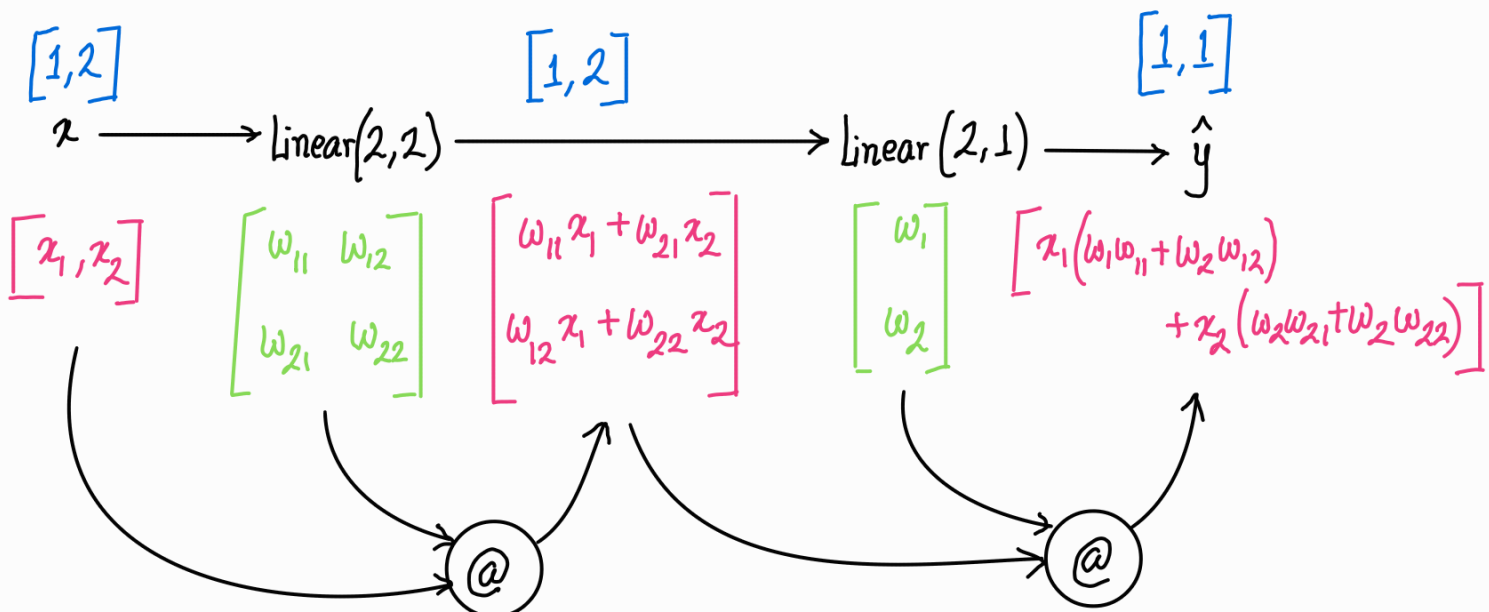
$h_2 = \text{Linear}_2(h_1) = h_1 \omega_2^T + b_2 \leftarrow [6]$

$\uparrow$
[6, 6] Blue indicates dimension

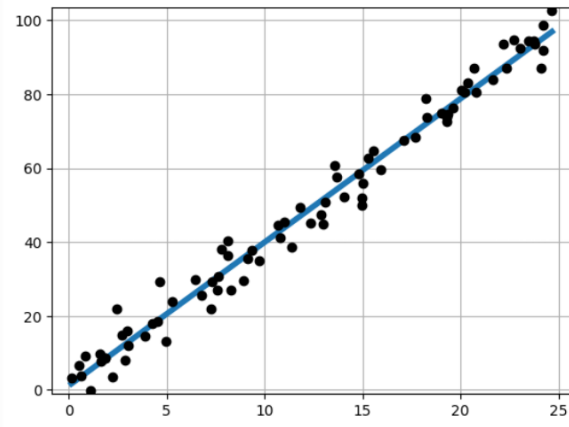
$\hat{y} = \text{Linear}_3(h_2) = h_2 \omega_3^T + b_3 \leftarrow [2]$

$\uparrow$
[2, 6]

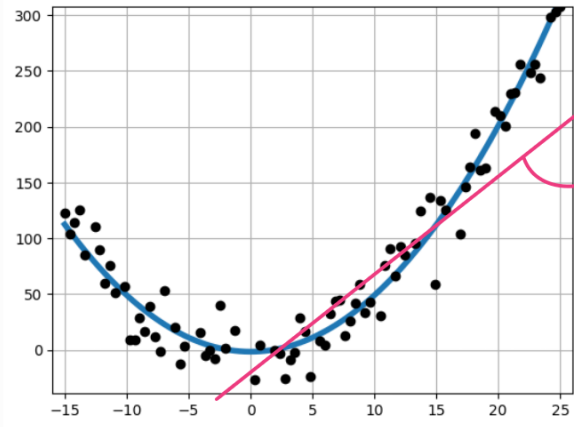
You can try out the operation and you will see that no matter how many Linear layers you may cascade, the output will never be a non-linear function of the input. Lets try one simple network. We will assume bias = False.



Without non-linearity in our system, Our system cannot fit on non-linear data.

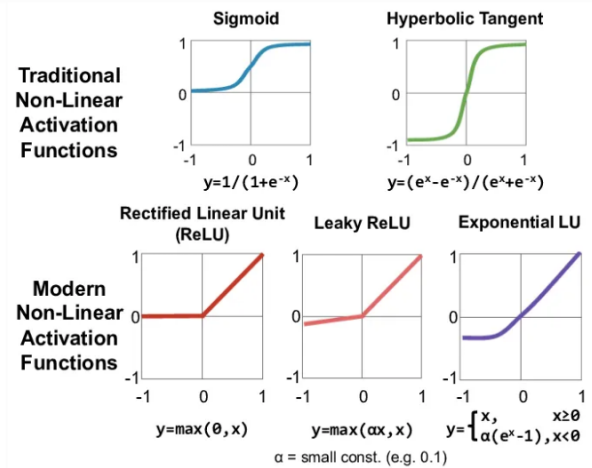
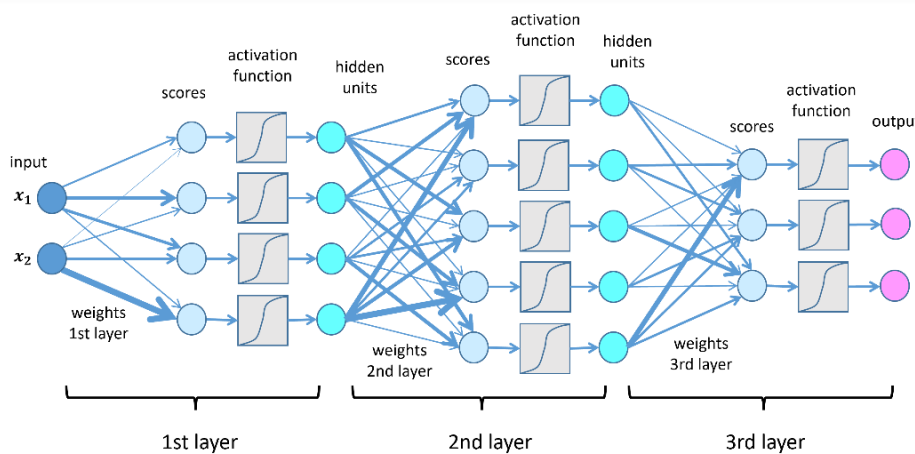


Linear data



Non-linear data

How do we add non-linearity to our neural networks? We use activation functions after our linear layers.



Example —

If $x =$

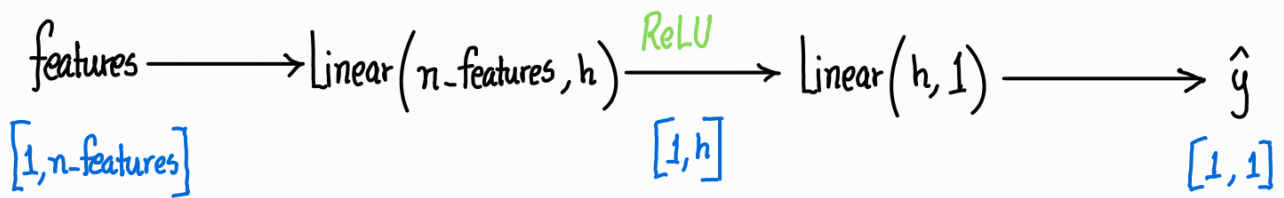
-2	2	5	-5
1	-2	3	-4
4	-6	-7	-1

, then what is $\text{ReLU}(x)$?

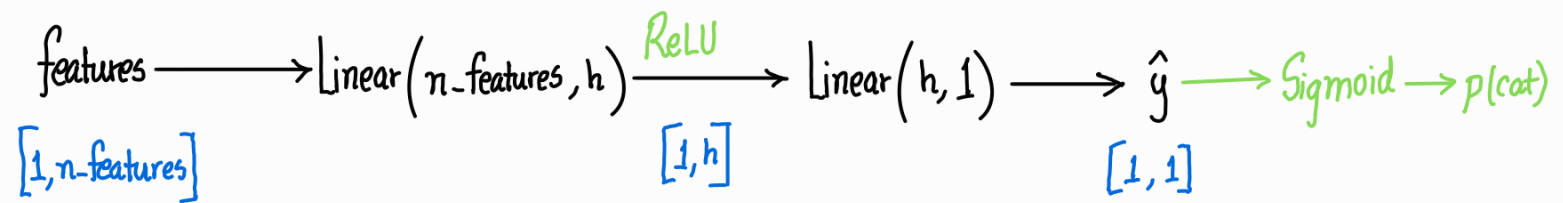
$\text{ReLU}(x) =$

0	2	5	0
1	0	3	0
4	0	0	0

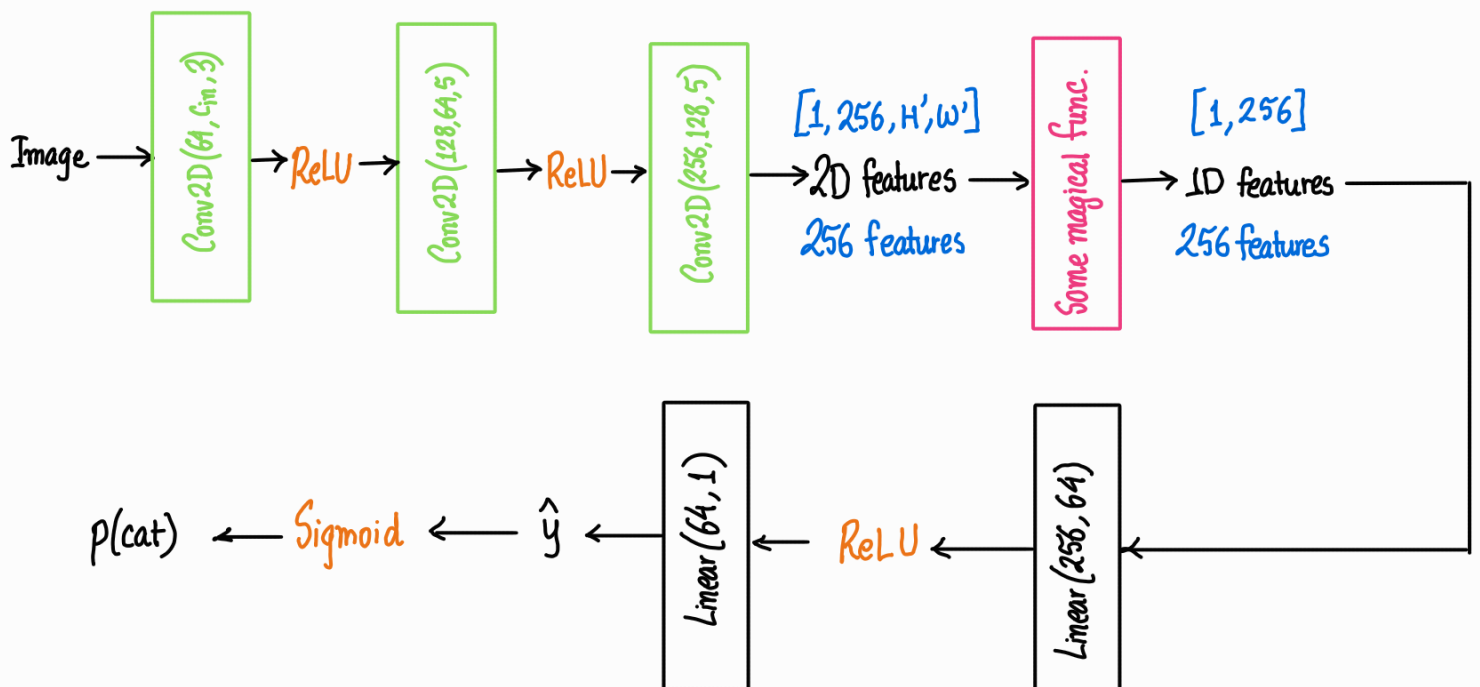
So, now we have a way to create the decision module. Let's assume we have a 1D featurespace from our convolutional network. We just need to predict the probability of the features belonging to the cat class or the dog class. Since there is only 2 classes, the image will be either a cat or a dog. This implies $p(\text{cat}) = 1 - p(\text{dog})$. So it is enough to predict one probability, $p(\text{cat})$.



But here $\hat{y} \in \mathbb{R}$ but probability $\in [0, 1]$. So, we have to apply sigmoid to convert continuous $\hat{y}$ to probability. $\hat{y}$ has a special name called logit.

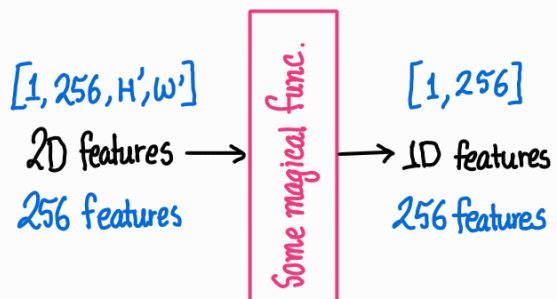


So now our whole system now looks like —



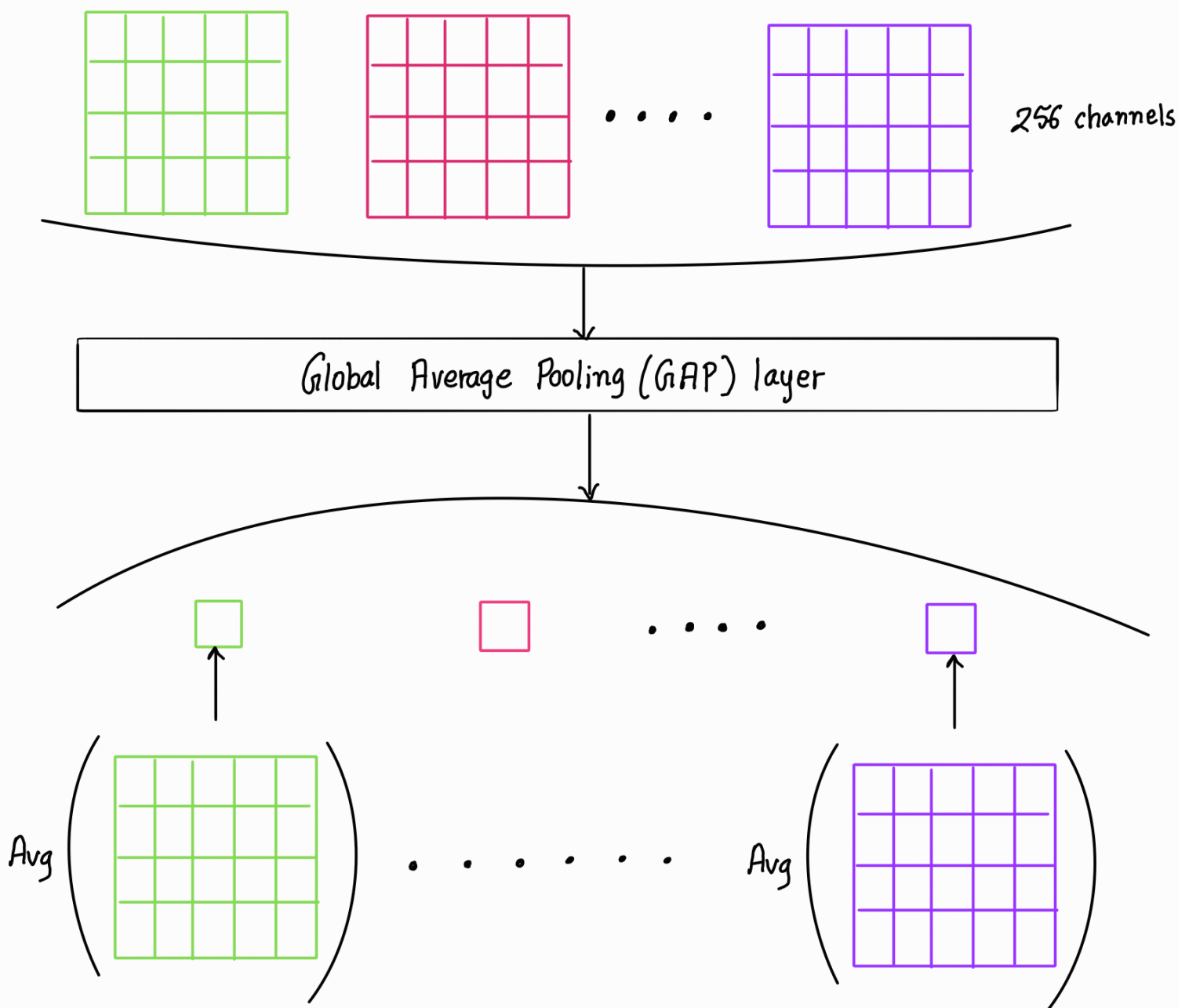
We added **ReLU** to our CNN part too because conv. layers are linear.

Now more on the magical layer —



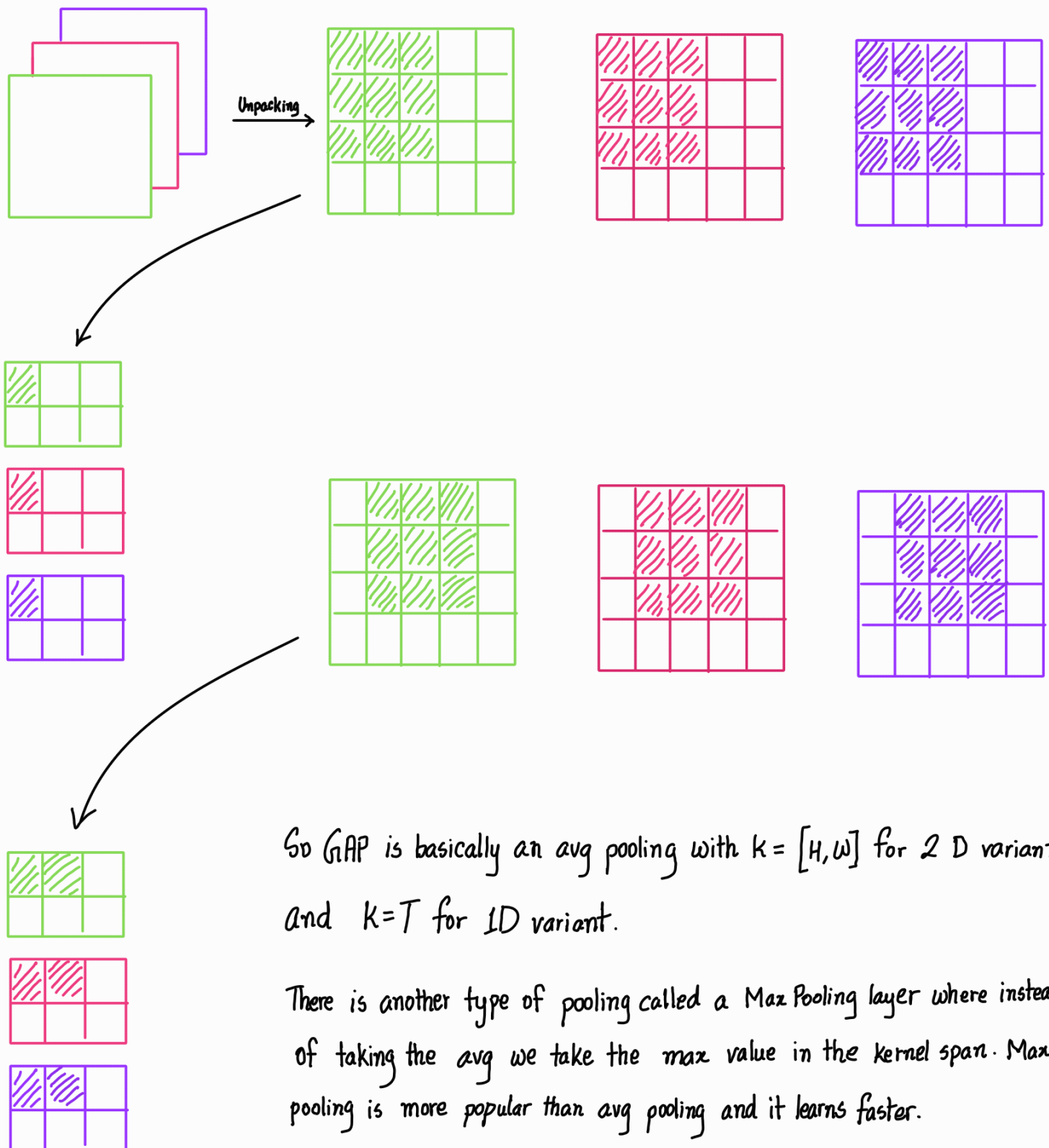
There are multiple ways to do this. But the most common method is a global average pooling (GAP) layer. What does the GAP layer do?

2D features —



Global Average Pooling?

The existence of global avg pooling indicates the existence of a regular avg pooling. Avg pooling uses an averaging kernel but applies cross-correlation channelwise. Lets see an example with a kernel, $k=3$ —



So GAP is basically an avg pooling with $k = [H, W]$ for 2D variant and $k = T$ for 1D variant.

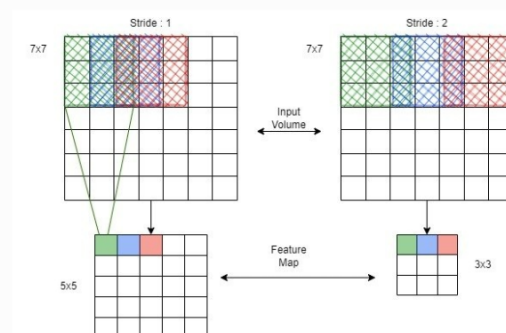
There is another type of pooling called a Max Pooling layer where instead of taking the avg we take the max value in the kernel span. Max pooling is more popular than avg pooling and it learns faster.

Max Pooling				Average Pooling			
29	15	28	184	31	15	28	184
0	100	70	38	0	100	70	38
12	12	7	2	12	12	7	2
12	12	45	6	12	12	45	6

2 x 2 pool size				2 x 2 pool size			
100	184			36	80		
12	45			12	15		

For reference you can follow this. Here stride = 2.

Stride is another hyperparameter for Conv. models.



Why use pooling layers?

1. Pooling layers increase the effective receptive field without increasing parameters.
2. Since we are collapsing the z (for 1D) and x, y (for 2D) dimensions at the end, we downscale the features slowly over the network instead of abruptly zeroing it out so that the model can adapt better during training. It also significantly cuts computation and RAM usage.

Conclusion—

Starting from the absolute bottom, we have built a model capable of distinguishing cats and dogs. We will not go through the learning process in details. It has already been discussed a bit in the lab hopefully. If not, it will be touched upon in the next lab session.

In the next class, we will discuss a bit on the dimension change through the network depending on different hyperparameters and we will close our discussion on convolution.